NP완전 문제

여러 선택을 바꿔 보며 조건에 맞는 답을 구합니다.

먼저 작은 예로 살펴봅니다

답을 찾기

여러 선택을 바꿔 보며 조건에 맞는 답을 구합니다.

답을 확인하기

이미 준 선택이 맞는지 계산만 합니다. 앞의 세 문제에서 이 차이를 봅니다.

9-1 제시된 부분집합 합 확인

구할 것

이미 골라 놓은 수들의 합이 목표와 같은지 확인합니다.

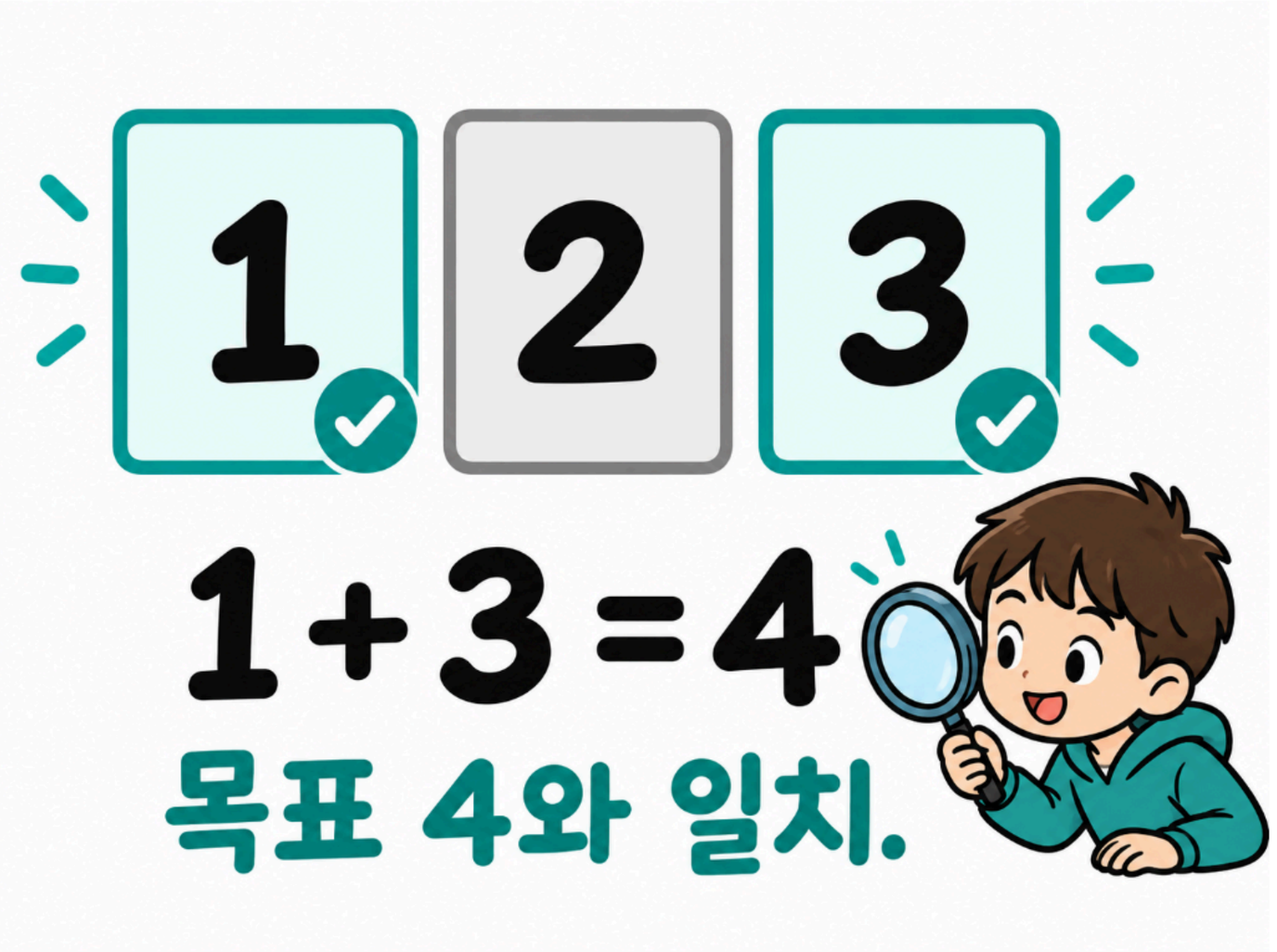
작은 예

a=[1,2,3], selected=[1,0,1], target=4입니다.
표시가 1인 첫째와 셋째 원소를 고른 상태입니다.

반환할 값

1+3=4이므로 맞으면 1을, 다르면 0을 반환합니다.

9-1 그림으로 보는 예



선택 표시 1,0,1
고른 값은 1과 3
합은 4
목표 4와 같으므로 1

9-1 하나씩 계산한 결과

배열의 값	선택 표시	이번에 더함	누적 합
1	1	1	1
2	0	0	1
3	1	3	4

함수가 새로 고를 수를 찾는 작업은 없습니다. 주어진 선택만 검사합니다.

9-1 계산 순서

합을 0으로 시작

선택한 원소가 없으면 합은 0입니다.

표시가 1일 때만 더함

selected[i]==1이면 a[i]를 합에 더합니다.

목표와 비교

최종 합과 target이 같으면 1, 다르면 0입니다.

9-1 변수와 코드의 뜻

코드	뜻
selected[i]	i번 원소를 골랐는지 표시한 0 또는 1
total += a[i]	선택한 원소의 값 더하기
total == target	선택된 값의 합이 목표와 같은지 비교

맞는 답을 제시받으면 확인만 하면 됩니다.

9-1 함수에 들어오는 값

함수

verify_subset_sum(a, selected, target)

입력 조건

원소 0~20개, 값 0~100, 목표 0~2000. 선택 표시는 같은 길이의 0/1 배열입니다.

9-1 Python

```
def verify_subset_sum(a, selected, target):
    total = 0
    for i in range(len(a)):
        if selected[i]:
            total += a[i]
    return int(total == target)
```

시간 O(n), 추가 공간 O(1)

9-1 C

```
int verify_subset_sum(const int a[], const int selected[],
    int n, int target) {
    int total=0;
    for (int i=0; i<n; ++i) if (selected[i]) total+=a[i];
    return total==target;
}
```

시간 O(n), 추가 공간 O(1)

9-1 다른 입력도 확인합니다

상황	답	이유
고른 수 없음, 목표 0	1	빈 합은 0
고른 수 없음, 목표 2	0	0과 2는 다름
예제의 목표를 5로 변경	0	고른 합은 여전히 4

맞는 답을 제시받으면 확인만 하면 됩니다.

9-2 제시된 정점 커버 확인

구할 것

고른 점들이 모든 선을 덮는지 확인합니다.

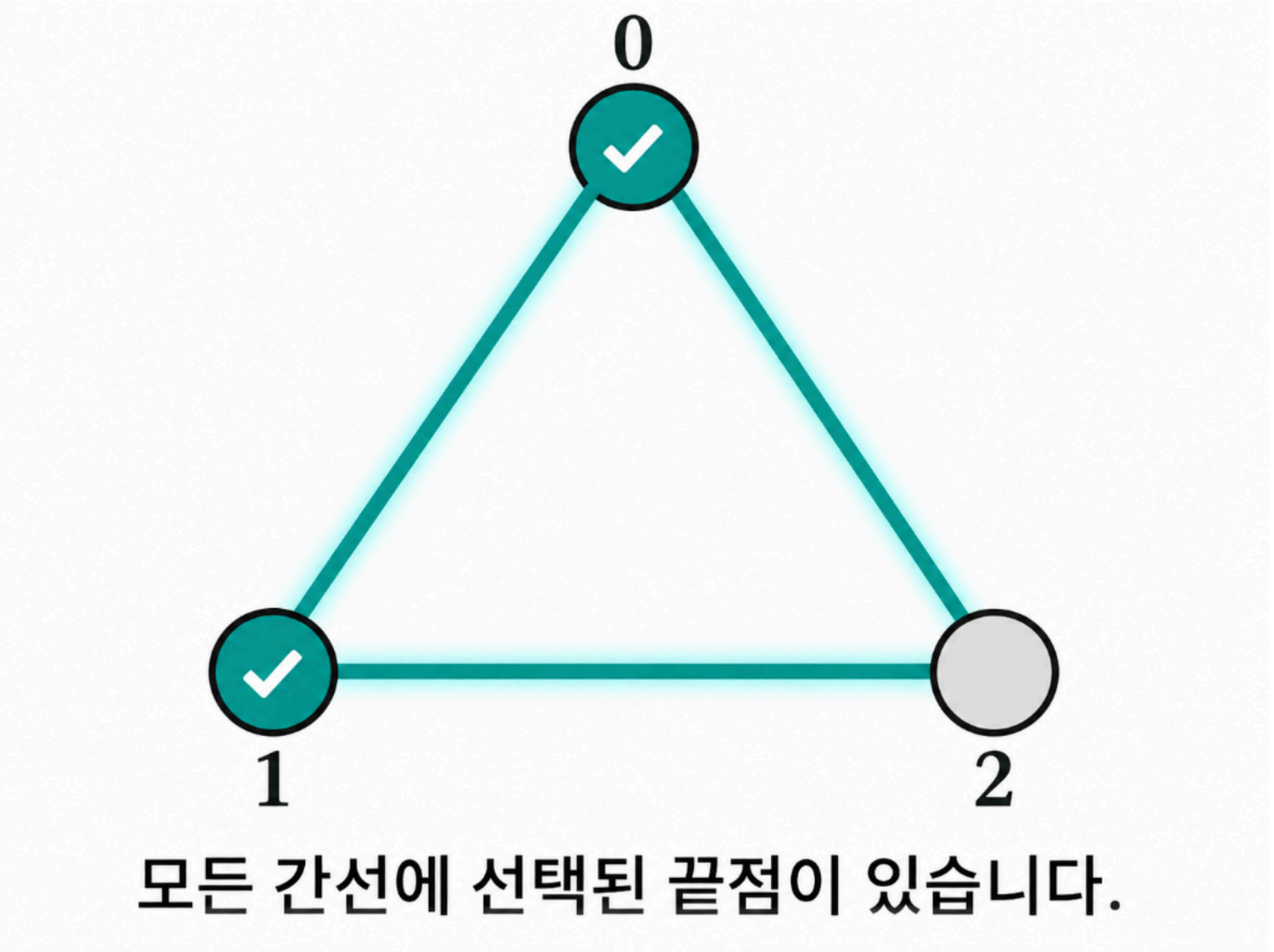
작은 예

삼각형의 점 0과 1을 골랐습니다.
선 0–1, 0–2, 1–2는 모두 고른 점과 닿아 있습니다.

반환할 값

모든 선에 고른 끝점이 하나 이상 있으면 1, 아니면 0을 반환합니다.

9-2 그림으로 보는 예



점 0과 1 선택
각 선의 양 끝 중
적어도 하나가 선택됨
모든 선을 덮었습니다.

9-2 하나씩 계산한 결과

검사할 선	끝점 중 고른 점	통과 여부
0–1	0과 1	통과
0–2	0	통과
1–2	1	통과

이처럼 모든 선에 닿도록 점을 고른 것을 정점 커버라고 부릅니다.

9-2 계산 순서

선 하나 확인

graph[u][v]==1인 두 점을 봅니다.

두 끝점 모두 안 골랐으면

이 선을 덮지 못했으므로 바로 0을 반환합니다.

끝까지 문제없으면

모든 선을 덮었으므로 1을 반환합니다.

9-2 변수와 코드의 뜻

코드	뜻
selected[i]	점 i를 고르면 1, 아니면 0
not selected[i] and not selected[j]	두 끝점 모두 고르지 않음
j를 i+1부터 검사	무방향 선 하나를 두 번 검사하지 않음

이 문제는 주어진 선택의 검사입니다. 최소 개수의 점을 찾을 필요는 없습니다.

9-2 함수에 들어오는 값

함수

verify_vertex_cover(graph, selected)

입력 조건

점 1~8개. 연결 표는 대칭인 0/1 표이며 대각선은 0입니다.

9-2 Python

```
def verify_vertex_cover(graph, selected):
    n = len(graph)
    for i in range(n):
        for j in range(i+1, n):
            if graph[i][j] and not selected[i] and not selected[j]:
                return 0
    return 1
```

인접 행렬 기준 시간 $O(n^2)$, 추가 공간 $O(1)$

9-2 C

```
int verify_vertex_cover(const int graph[][8], int n,
    const int selected[]) {
    for (int i=0; i<n; ++i)
        for (int j=i+1; j<n; ++j)
            if (graph[i][j] && !selected[i] && !selected[j])
                return 0;
    return 1;
}
```

인접 행렬 기준 시간 $O(n^2)$, 추가 공간 $O(1)$

9-2 다른 입력도 확인합니다

상황	답	이유
선이 하나도 없음	1	못 덮은 선이 없음
삼각형에서 점0만 선택	0	1–2 선을 못 덮음
삼각형에서 점0,1 선택	1	세 선을 모두 덮음

이 문제는 주어진 선택의 검사입니다. 최소 개수의 점을 찾을 필요는 없습니다.

9-3 제시된 해밀턴 순회 확인

구할 것

주어진 순서가 모든 점을 한 번씩 지나 출발점으로 돌아오는지 검사합니다.

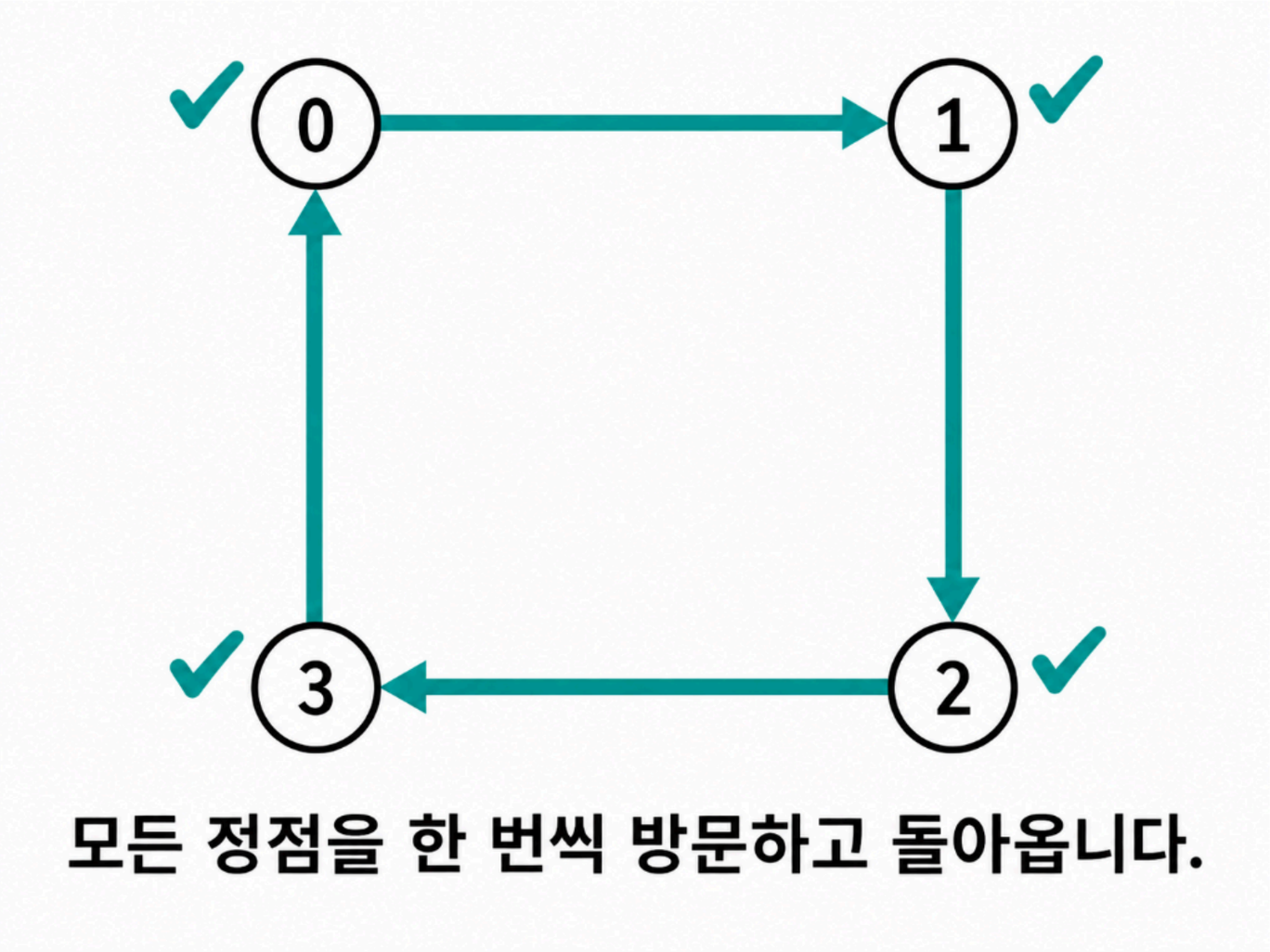
작은 예

정점이 0,1,2,3이고 path=[0,1,2,3]입니다.
0–1, 1–2, 2–3뿐 아니라 3–0 선도 있어야 합니다.

반환할 값

빠진 점·중복 점·없는 연결이 모두 없으면 1을 반환합니다.

9-3 그림으로 보는 예



0,1,2,3을 한 번씩
이웃한 점의 선 확인
마지막 3–0도 확인
모두 맞으면 1

9-3 하나씩 계산한 결과

검사 순서	확인할 내용	통과 조건
1	점 번호	모두 0..n−1 안에 있음
2	방문 횟수	같은 점이 두 번 나오지 않음
3	중간 연결	0–1, 1–2, 2–3 존재
4	귀환 연결	3–0 존재

모든 점을 한 번씩 방문해 돌아오는 순회를 해밀턴 순회라고 부릅니다.

9-3 계산 순서

방문 표시 준비

seen 배열을 전부 0으로 만듭니다.

번호와 중복 확인

범위 밖이거나 이미 본 번호면 0을 반환합니다.

모든 이동 확인

이웃한 두 점과 마지막·첫 점 사이에 선이 있는지 확인합니다.

9-3 변수와 코드의 뜻

변수	뜻
seen[v]	점 v가 path에서 이미 나왔으면 1
path[i]	이번 점
path[(i+1)%n]	다음 점. 마지막에서는 첫 점
graph[u][v]	이 이동에 선이 있으면 1

경로를 새로 만드는 작업 없이, 제시된 경로를 그대로 검사합니다.

9-3 함수에 들어오는 값

함수

verify_hamiltonian(graph, path)

입력 조건

점 3~8개. 연결 표는 대칭인 0/1 표. path의 길이는 점의 수와 같습니다.

9-3 Python

```
def verify_hamiltonian(graph, path):
    n = len(graph)
    seen = [False]*n
    for v in path:
        if v < 0 or v >= n or seen[v]:
            return 0
        seen[v] = True
    for i in range(n):
        if not graph[path[i]][path[(i+1)%n]]:
            return 0
    return 1
```

행렬이 주어졌을 때 검사 시간 O(n), seen 배열 O(n)

9-3 C

```
int verify_hamiltonian(const int graph[][8], int n,
    const int path[]) {
    int seen[8]={0};
    for (int i=0; i<n; ++i) {
        int v=path[i];
        if (v<0 || v>=n || seen[v]) return 0;
        seen[v]=1;
    }
    for (int i=0; i<n; ++i)
        if (!graph[path[i]][path[(i+1)%n]]) return 0;
    return 1;
}
```

행렬이 주어졌을 때 검사 시간 O(n), seen 배열 O(n)

9-3 다른 입력도 확인합니다

상황	답	이유
[0,1,1,3]	0	1 중복, 2 빠짐
[0,1,2,9]	0	9는 범위 밖
3–0 선이 없는 [0,1,2,3]	0	출발점으로 못 돌아옴

경로를 새로 만드는 작업 없이, 제시된 경로를 그대로 검사합니다.

9-4 작은 3-CNF 식의 만족 가능성

구할 것

참·거짓 값을 골라 모든 조건을 만족시킬 수 있는지 확인합니다.

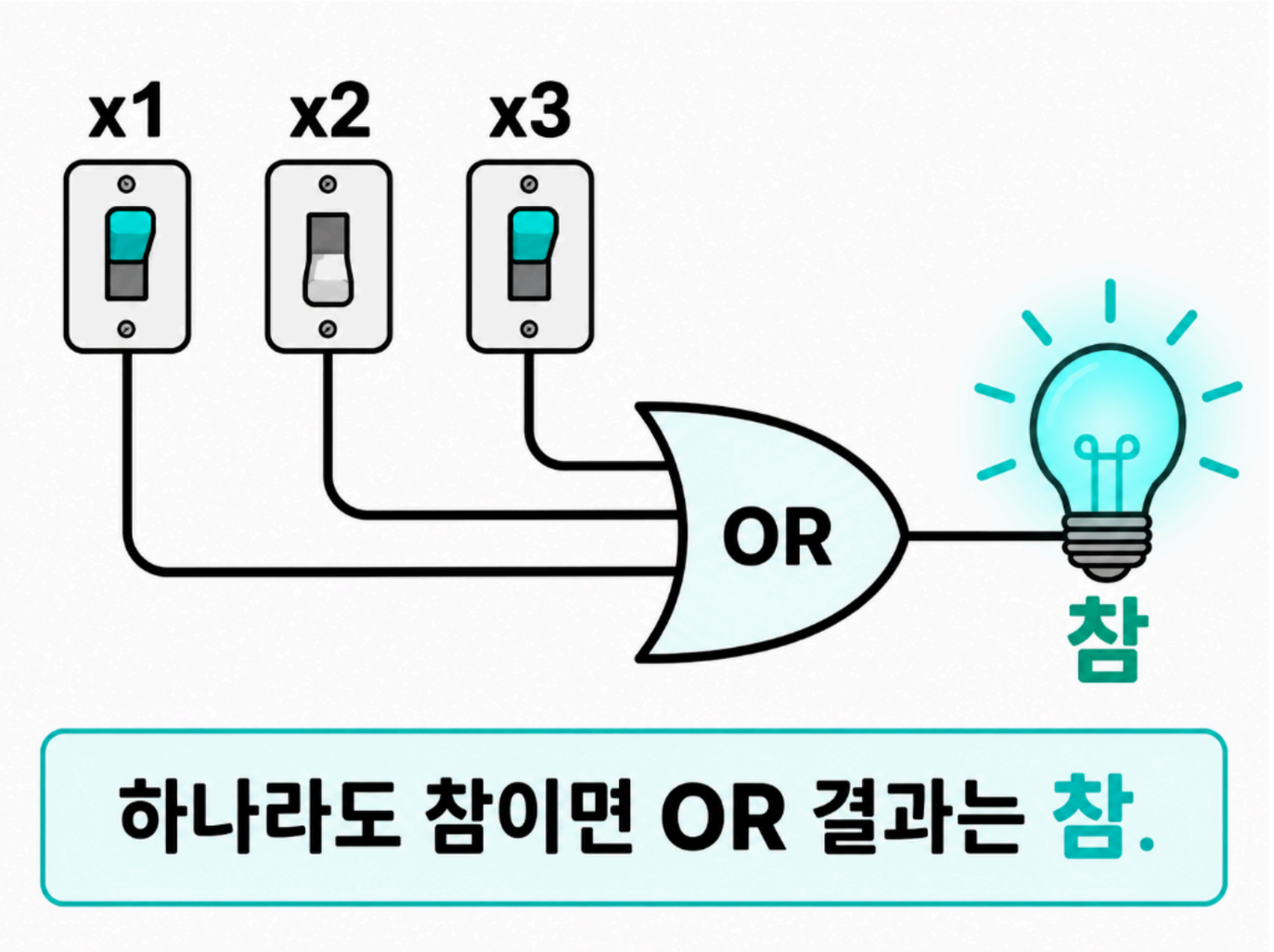
작은 예

x_1은 0(거짓) 또는 1(참) 중 하나입니다.
조건 [1,1,1]은 x_1이 참이어야 통과합니다.

반환할 값

모든 조건을 통과하는 값 배정이 하나라도 있으면 1을 반환합니다.

9-4 그림으로 보는 예



한 묶음 안에서는
하나라도 참이면 통과
모든 묶음이 통과하면
전체 조건을 만족합니다.

9-4 하나씩 계산한 결과

값 배정	조건 [1,1,1]	조건 [−1,−1,−1]	둘 다 통과?
$x_1 = 0$	실패	통과	아니요
$x_1 = 1$	통과	실패	아니요

위 두 조건을 동시에 만족하는 배정은 없으므로 이 예의 답은 0입니다.

0과 1 배정을 정수로 표현합니다

변수 두 개이면 가능한 배정은 네 개입니다.

mask	두 비트	x_2	x_1
0	00	0	0
1	01	0	1
2	10	1	0
3	11	1	1

오른쪽 끝이 x_1입니다. 0부터 3까지 보면 네 배정을 모두 봅니다.

코드의 비트 연산을 한 번 계산합니다

mask=5는 이진수 101, 즉 x_3=1, x_2=0, x_1=1입니다.

확인할 변수	오른쪽으로 이동	끝 비트만 읽기	값
x_1	101 >> 0 = 101	101 & 1	1
x_2	101 >> 1 = 10	10 & 1	0
x_3	101 >> 2 = 1	1 & 1	1

음수 항이면 읽은 값을 1−value로 뒤집습니다.

9-4 계산 순서

값 배정 하나 시도

변수마다 0 또는 1을 정합니다.

조건 묶음 확인

각 묶음의 세 값 중 하나라도 1인지 확인합니다.

모든 묶음 통과

그 배정으로 성공입니다. 없으면 다음 배정을 시도합니다.

9-4 변수와 코드의 뜻

입력 표현	뜻
양수 2	x_2의 값을 그대로 사용
음수 –2	x_2가 0이면 1, 1이면 0으로 뒤집음
[1,–2,3]	x_1, 반대 x_2, x_3 중 하나라도 참이면 통과
clauses	이런 세 값짜리 조건 묶음들의 목록

세 항을 OR로 묶은 조건들을 AND로 연결한 식을 3-CNF라고 부릅니다.

9-4 함수에 들어오는 값

함수

satisfiable_3cnf(n, clauses)

입력 조건

변수 1~12개, 조건 묶음 0~30개. 묶음마다 항 3개이며 1≤|항|≤n입니다.

9-4 Python

```
def satisfiable_3cnf(n, clauses):
    for mask in range(1 << n):
        satisfied = True
        for clause in clauses:
            clause_ok = False
            for literal in clause:
                value = (mask >> (abs(literal)-1)) & 1
                if literal < 0: value = 1-value
                if value: clause_ok = True
            if not clause_ok:
                satisfied = False
                break
        if satisfied: return 1
    return 0
```

시간 $O(2^n \cdot \max(1,m))$, 배정 비트마스크 외 추가 공간 $O(1)$

9-4 C (1/2)

```
int satisfiable_3cnf(int n, const int clauses[][3], int m) {
    for (int mask=0; mask<(1<<n); ++mask) {
        int ok=1;
        for (int i=0; i<m; ++i) {
            int clause_ok=0;
            for (int j=0; j<3; ++j) {
                int lit=clauses[i][j], k=lit<0?-lit:lit;
                int value=(mask>>(k-1))&1;
                if (lit<0) value=1-value;
                if (value) clause_ok=1;
```

다음 쪽에서 이어집니다.

9-4 C (2/2)

```
            }
            if (!clause_ok) {
                ok=0;
                break;
            }
        }
        if (ok) return 1;
    }
    return 0;
}
```

시간 $O(2^n \cdot \max(1,m))$, 배정 비트마스크 외 추가 공간 $O(1)$

9-4 다른 입력도 확인합니다

상황	답	이유
조건 묶음이 없음	1	위반할 조건이 없음
[1,1,1]만 있음	1	x_1=1이면 통과
위 두 반대 조건을 함께 줌	0	0과 1 모두 실패

세 항을 OR로 묶은 조건들을 AND로 연결한 식을 3-CNF라고 부릅니다.

찾기는 오래 걸려도 확인은 간단할 수 있습니다

부분집합 예

어떤 수를 고를지 찾을 때는 많은 선택을 봅니다. 선택 표시를 주면 합만 계산합니다.

P와 NP

P는 입력 크기의 다항 시간 안에 풀 수 있는 결정 문제, NP는 예라는 답의 증거를 다항 시간 안에 확인할 수 있는 문제입니다.

NP완전

NP에 속하면서 NP의 모든 문제를 다항 시간 변환으로 옮길 수 있는 문제입니다.

9장의 계산을 한 문장으로

문제	구하는 값
9-1	이미 골라 놓은 수들의 합이 목표와 같은지 확인합니다.
9-2	고른 점들이 모든 선을 덮는지 확인합니다.
9-3	주어진 순서가 모든 점을 한 번씩 지나 출발점으로 돌아오는지 검사합니다.
9-4	참·거짓 값을 골라 모든 조건을 만족시킬 수 있는지 확인합니다.

여러 선택을 바꿔 보며 조건에 맞는 답을 구합니다.